

Computer Networks (CS-241)

Aqsa Safdar
Department of Computer Science,
University of Wah

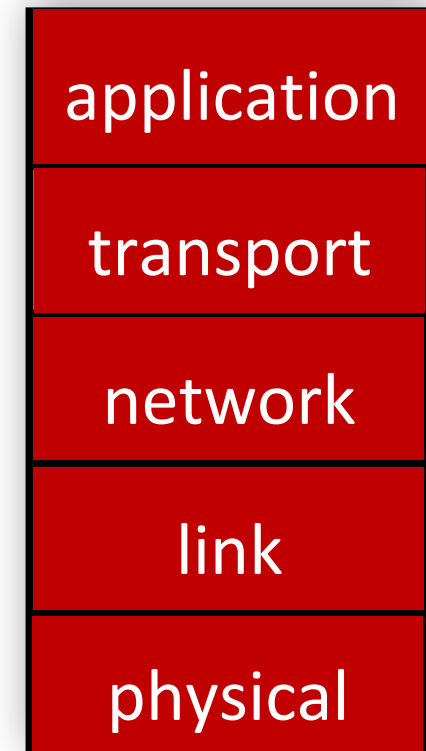
Adapted From:

Computer Networking: A Top-Down Approach

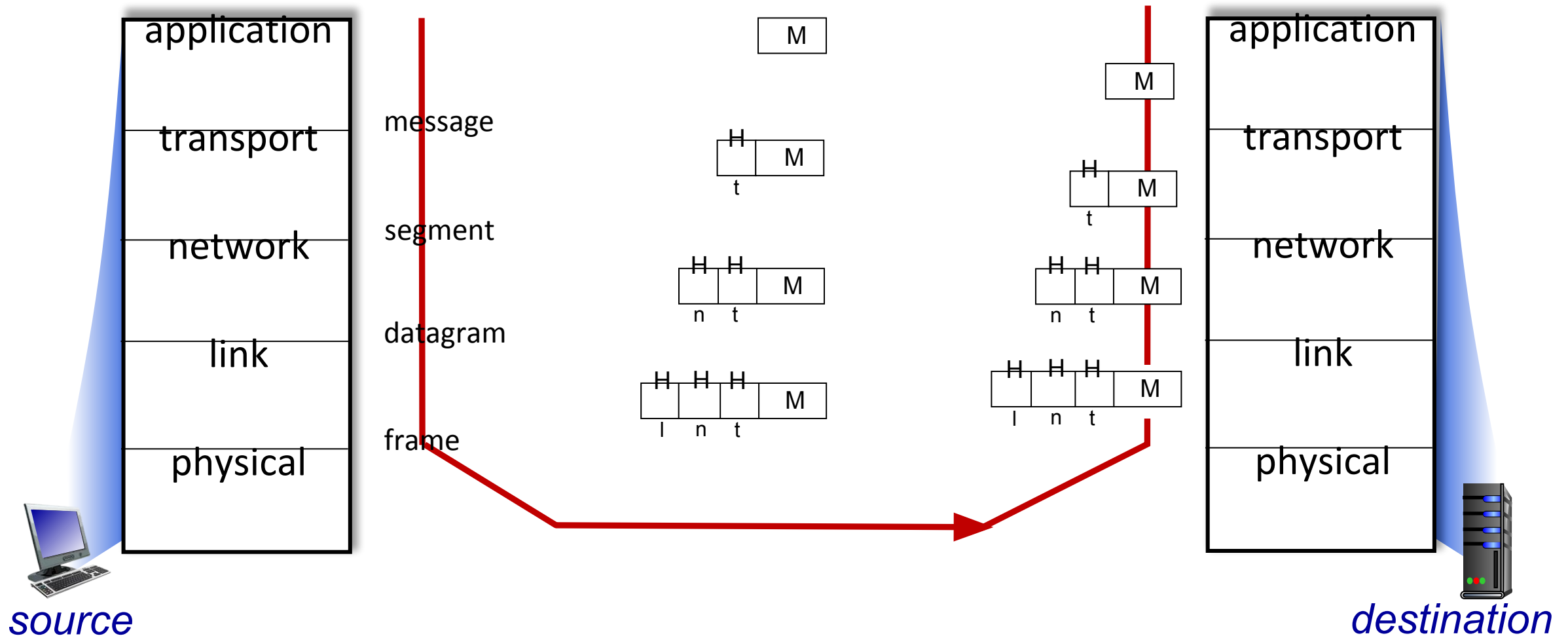
8th edition: Jim Kurose, Keith Ross Pearson, 2020

Recapitulation: Layered TCP/IP stack (1/3)

- *application*: supporting network applications
 - HTTP, IMAP, SMTP, DNS
- *transport*: process-process data transfer
 - TCP, UDP
- *network*: routing of datagrams from source to destination
 - IP, routing protocols
- *link*: data transfer between neighboring network elements
 - Ethernet, 802.11 (WiFi), PPP
- *physical*: bits “on the wire”



Recapitulation: Services, Layering and Encapsulation (2/3)



Recapitulation: Application layer (3/3)

The Application Layer is the topmost layer of the OSI (Open Systems Interconnection) model. It provides services and interfaces that allow end users and applications to access the network. Key responsibilities of application layer are mentioned below.

Network services for applications: Provides protocols and services for user-level network interactions.

Data formatting and translation: Converts data from application formats into network formats (and vice versa).

Resource sharing and remote access: Allows sharing of files, printers, and other network resources.

User authentication and privacy: Supports security features such as logins, encryption, and session control.

Transport layer: overview

Our goal:

- understand principles behind transport layer services:
 - multiplexing, demultiplexing
 - reliable data transfer
 - flow control
 - congestion control
- learn about Internet transport layer protocols:
 - UDP: connectionless transport
 - TCP: connection-oriented reliable transport
 - TCP congestion control

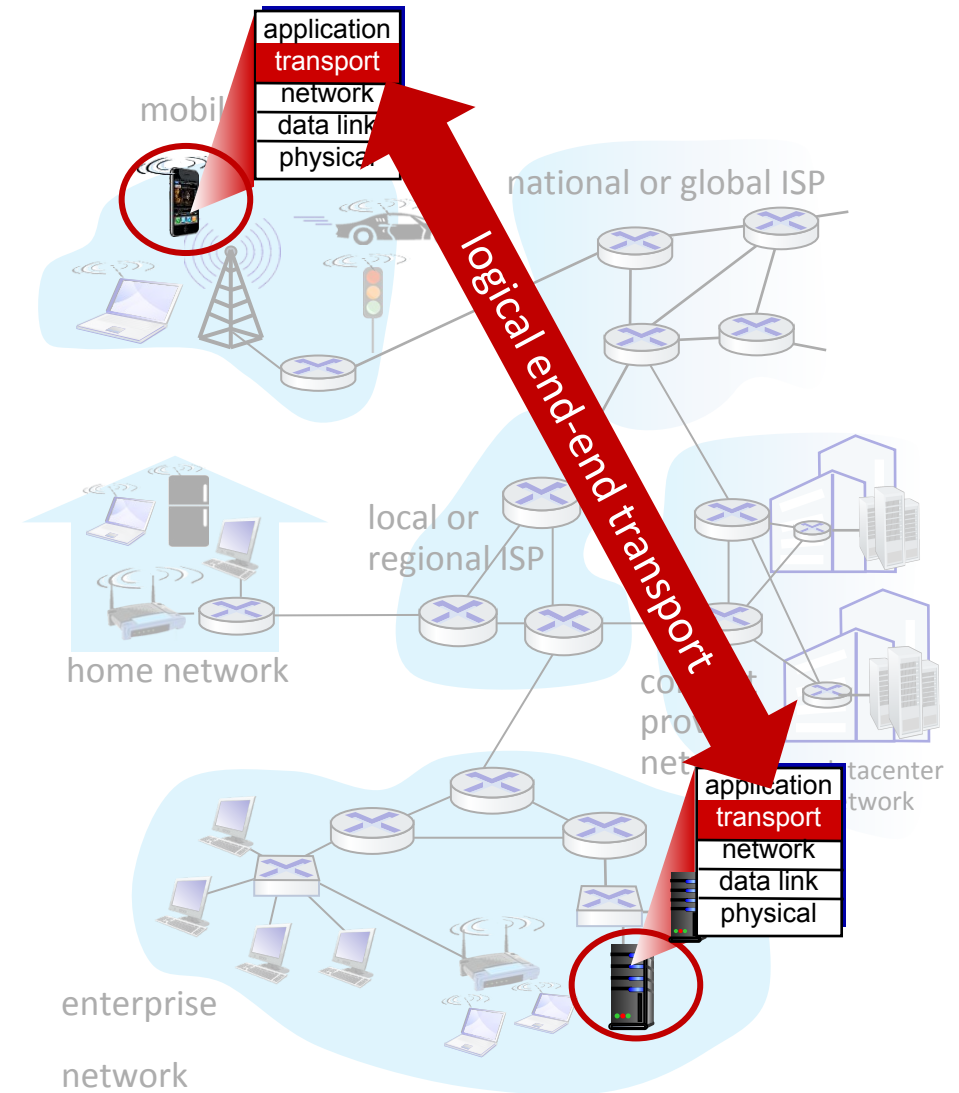
Transport layer: roadmap

- Transport-layer services
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality



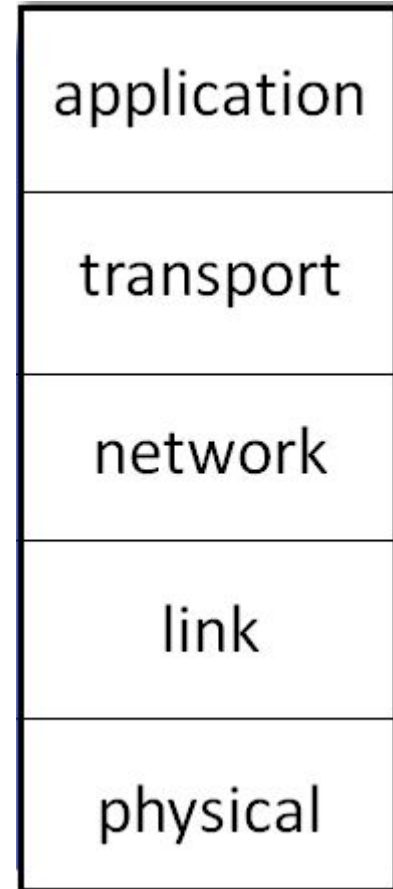
Transport services and protocols

- provide *logical communication* between application processes running on different hosts
- transport protocols actions in end systems:
 - sender: breaks application messages into *segments*, passes to network layer
 - receiver: reassembles segments into messages, passes to application layer
- two transport protocols available to Internet applications
 - TCP, UDP



Transport services and protocols

- The Transport Layer provides communication between processes running on different hosts. It sits above the Network Layer.
- **Services of Transport layer**
 1. **Process-to-process delivery:** Network layer provides host-to-host delivery, but transport layer delivers data to the correct application (e.g., browser, email, video call).
 2. **Multiplexing and De-multiplexing:**
 - **Multiplexing:** The transport layer collects data from multiple applications, adds transport layer headers (with port numbers), and passes them to the network layer as separate segments. This allows many applications to share the same network connection.
 - **De-multiplexing:** The transport layer receives segments from the network layer, checks the destination port numbers, and delivers each segment to the correct application process (e.g., web browser, email, etc.).



Transport services and protocols

3. **Error control:** Detects corrupted data using checksums (error-detection technique used to ensure data integrity during transmission).
4. **Reliable data transfer (in reliable protocols):**
Ensures data arrives without loss, without duplication, and in order.
5. **Flow control:** Prevents sender from overwhelming the receiver.
6. **Congestion control:** Prevents the network from being overloaded.

Transport Layer Protocols:

UDP (User Datagram Protocol) – unreliable, no connection.

TCP (Transmission Control Protocol) – reliable, connection-oriented.

Transport vs. network layer services and protocols

- **Network layer:** logical communication between *hosts*
 - No guarantee of timing, order, or reliability. IP protocol works here.
- **Transport layer:** logical communication between *processes*
 - Builds additional features on top of IP:
 - reliability (TCP)
 - flow control
 - congestion control

Transport layer depends on network layer for routing and delivery, but does not rely on it for reliability.

household

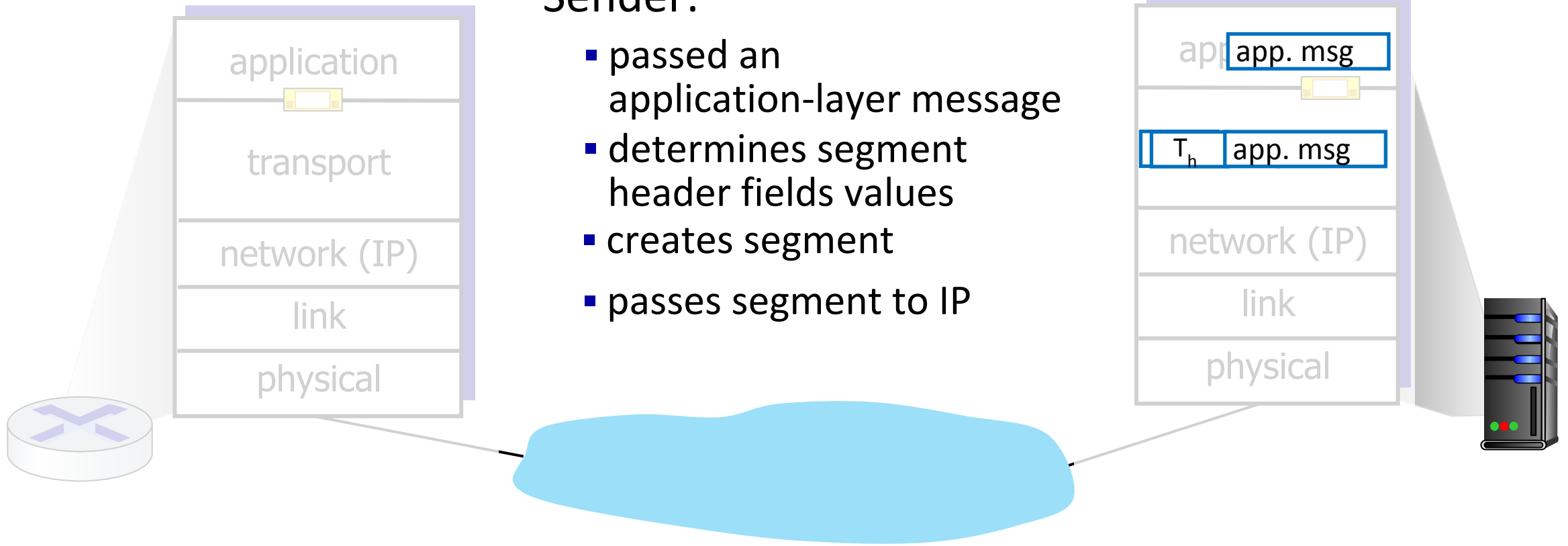
analogy:
12 kids in Mr. X's house
sending letters to 12 kids in
Mr. Y's house:

- hosts = houses
- processes = kids
- app messages = letters in envelopes
- transport protocol = Ann and Bill who demux to in-house siblings
- network-layer protocol = postal service

Transport Layer Actions

Sender:

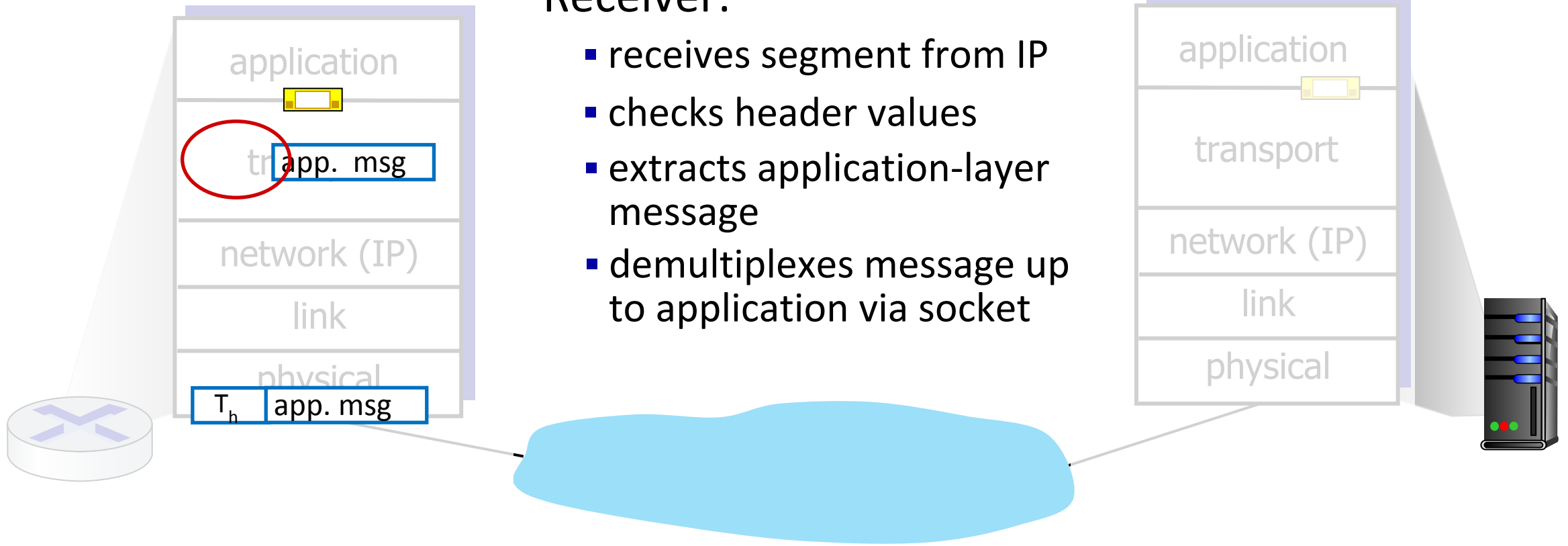
- passed an application-layer message
- determines segment header fields values
- creates segment
- passes segment to IP



Transport Layer Actions

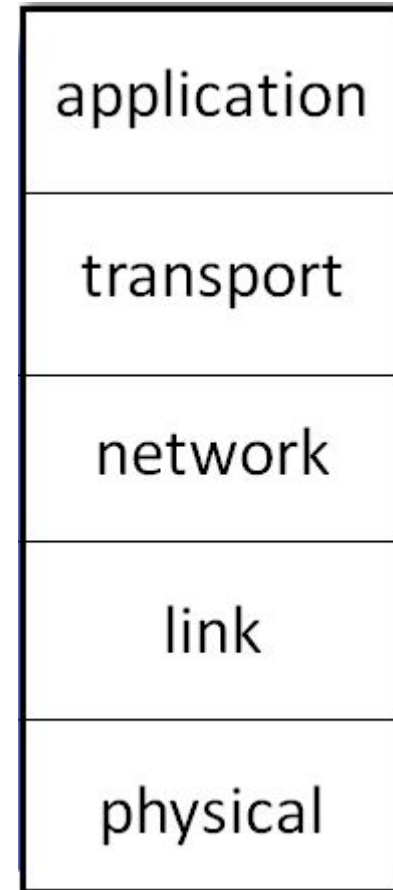
Receiver:

- receives segment from IP
- checks header values
- extracts application-layer message
- demultiplexes message up to application via socket



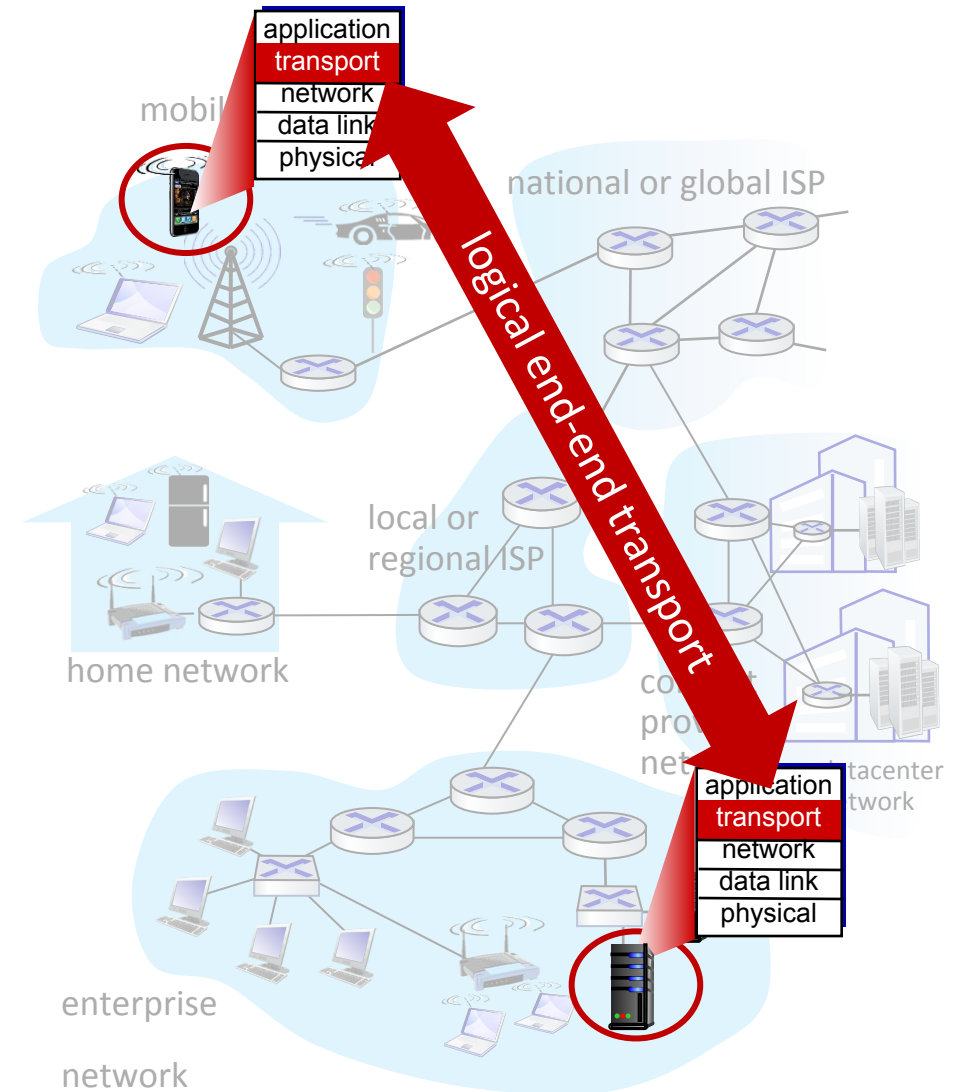
Overview of the Transport Layer in the Internet

Feature	Description
Layer Position	It is the 4th layer in the TCP/IP model (between the Application Layer and Network Layer).
Main Function	Provides end-to-end communication between processes (applications) running on different hosts.
Purpose	Ensures that data is delivered reliably, in order, and to the correct application on the receiving device.
Key Tasks / Functions	• Segmentation and Reassembly – Divides large data from applications into smaller segments and reassembles them at the receiver. • Multiplexing and Demultiplexing – Uses port numbers to manage multiple application connections simultaneously. • Connection Control – Establishes, maintains, and terminates logical connections. • Flow Control – Prevents a fast sender from overwhelming a slow receiver. • Error Control (Checksum) – Detects and sometimes corrects transmission errors.
Main Protocols	TCP (Transmission Control Protocol) – Reliable, connection-oriented communication (used for web, email, file transfer). UDP (User Datagram Protocol) – Unreliable, connectionless communication (used for streaming, gaming, VoIP).
Addressing	Uses Port Numbers (e.g., port 80 for HTTP, 25 for SMTP) to identify specific application processes.
Examples of Services	• Sending an email (SMTP over TCP) • Viewing a webpage (HTTP over TCP) • Video calling or streaming (RTP over UDP)



Two principal Internet transport protocols

- **TCP:** Transmission Control Protocol
 - reliable, in-order delivery
 - congestion control
 - flow control
 - connection setup
- **UDP:** User Datagram Protocol
 - unreliable, unordered delivery
 - no-frills extension of “best-effort” IP
- services not available:
 - delay guarantees
 - bandwidth guarantees



Chapter 3: roadmap

- Transport-layer services
- **Connectionless transport: UDP**
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality



UDP: User Datagram Protocol

- UDP provides the minimum transport-layer functionality. It does not add extra features like reliability or flow control.
- UDP provides best effort delivery, meaning: It tries to deliver data, but makes no guarantees.
- UDP does not ensure reliability, ordering, or protection against duplication.
- Skype, WhatsApp Calls, YouTube Live, Netflix, Multicast, DNS, DHCP, RIP etc.
 - *connectionless*:
 - no handshaking between UDP sender, receiver
 - each UDP segment handled independently of others

Why is there a UDP?

- no connection establishment (which can add RTT delay)
- simple: no connection state at sender, receiver
- small header size
- no congestion control
 - UDP can blast away as fast as desired!
 - can function in the face of congestion

UDP: User Datagram Protocol

- UDP use:
 - streaming multimedia apps (loss tolerant, rate sensitive)
 - DNS
 - SNMP
 - HTTP/3
- if reliable transfer needed over UDP (e.g., HTTP/3):
 - add needed reliability at application layer
 - add congestion control at application layer

UDP: User Datagram Protocol [RFC 768]

INTERNET STANDARD

RFC 768

J. Postel

ISI

28 August 1980

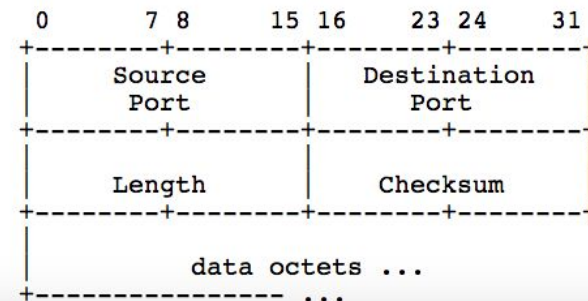
User Datagram Protocol

Introduction

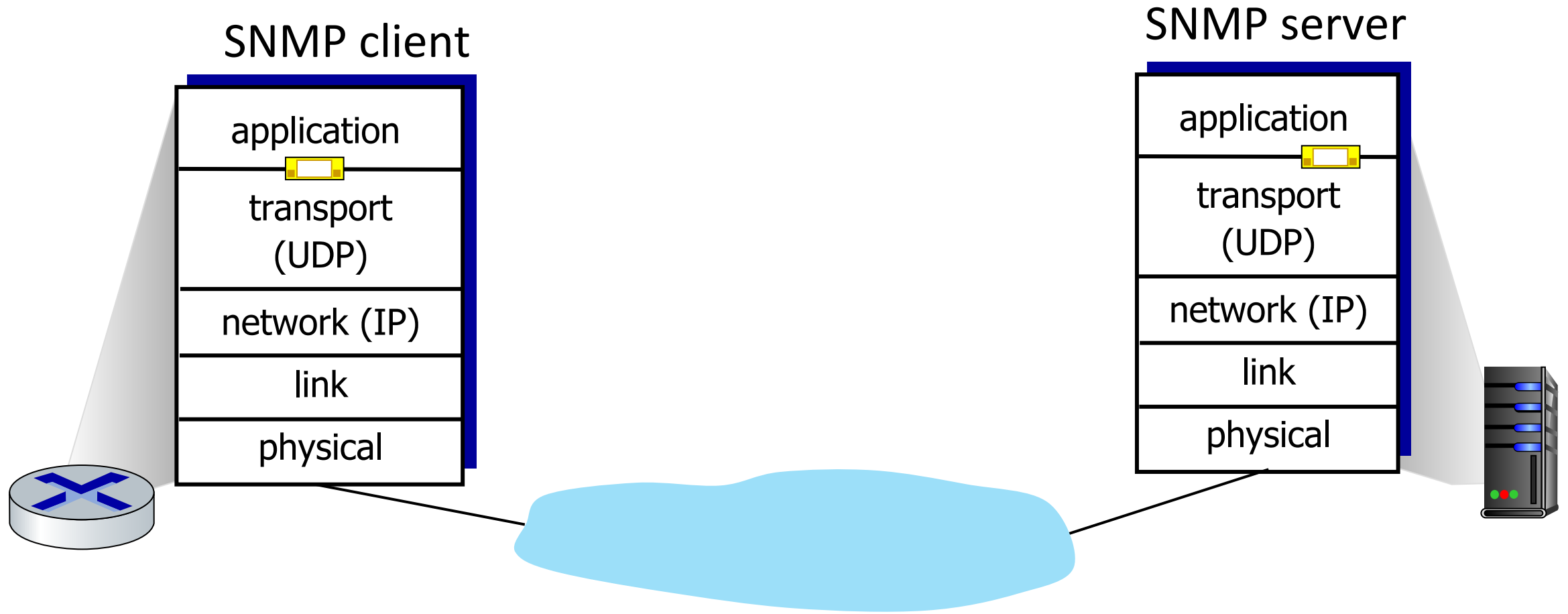
This User Datagram Protocol (UDP) is defined to make available a datagram mode of packet-switched computer communication in the environment of an interconnected set of computer networks. This protocol assumes that the Internet Protocol (IP) [1] is used as the underlying protocol.

This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed. Applications requiring ordered reliable delivery of streams of data should use the Transmission Control Protocol (TCP) [2].

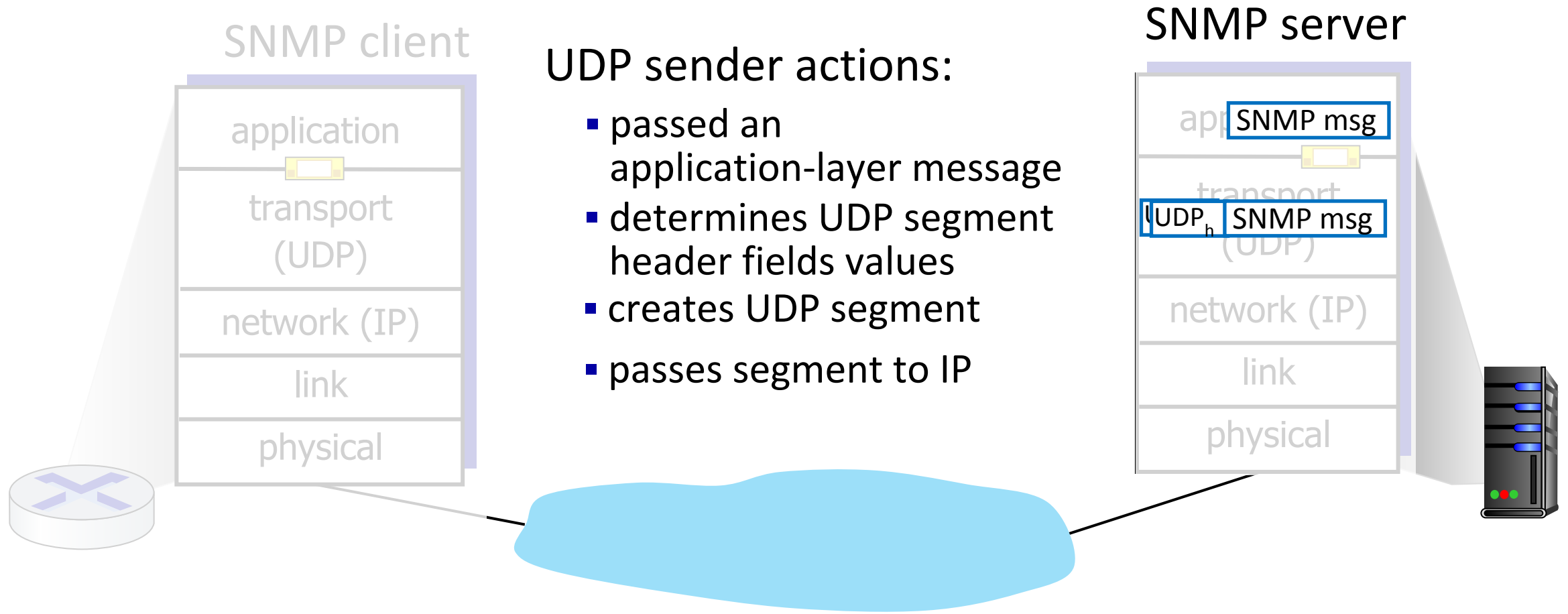
Format



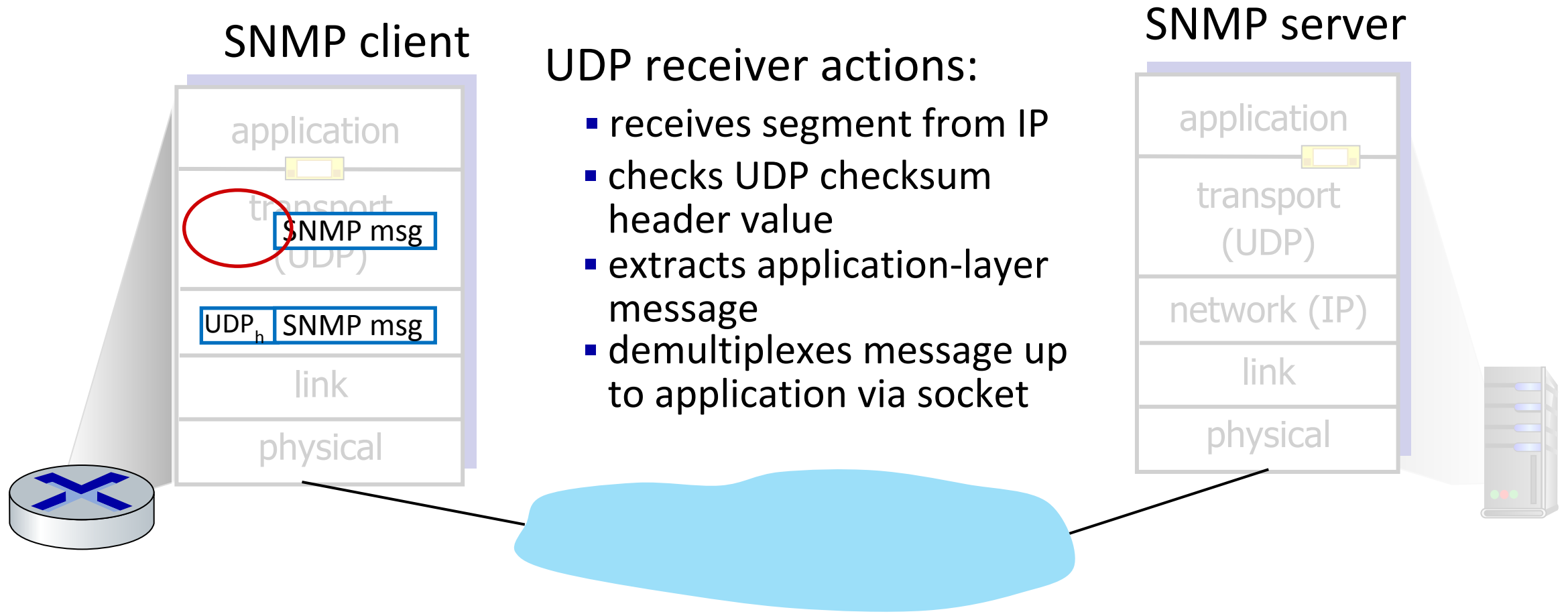
UDP: Transport Layer Actions



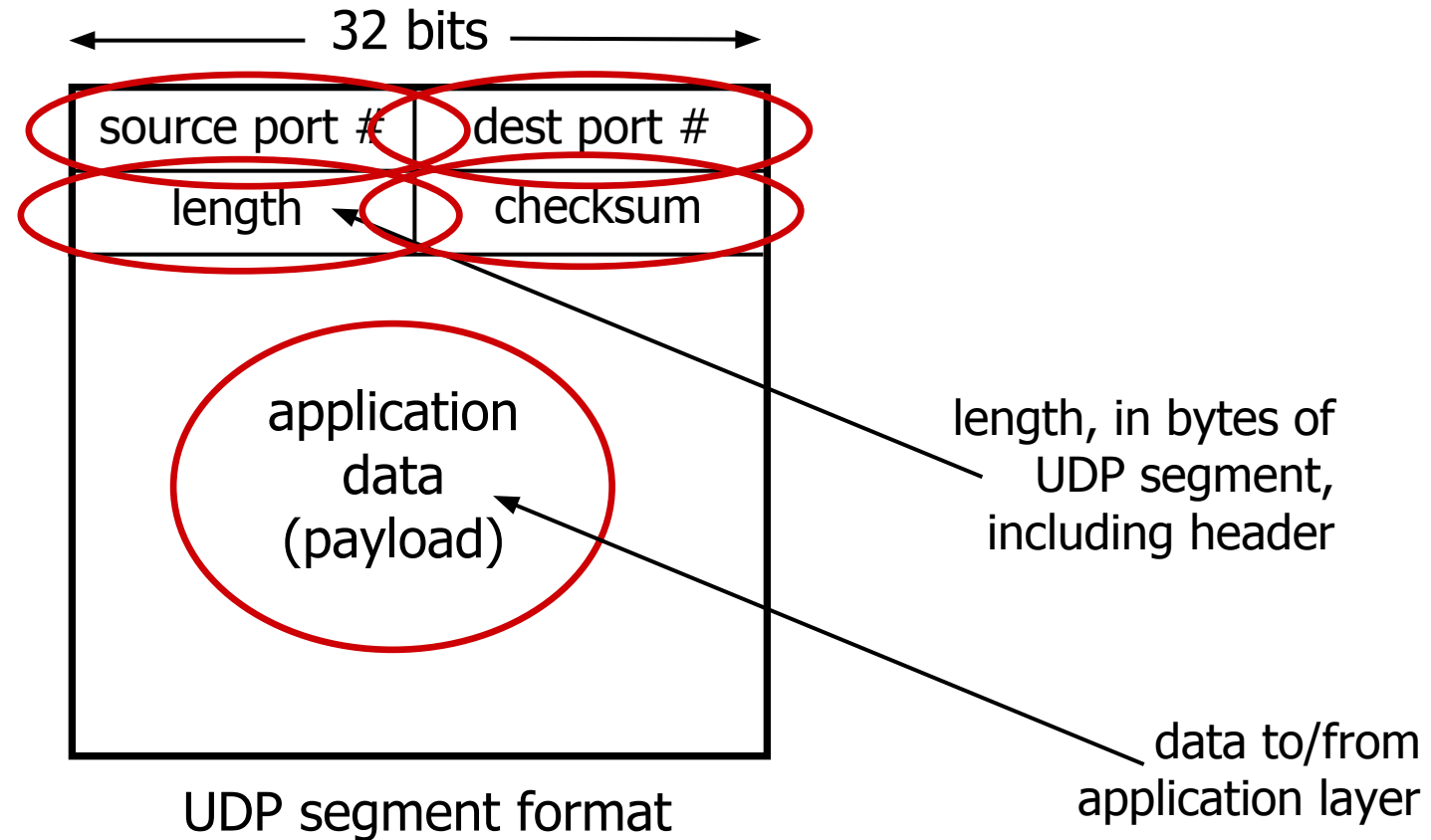
UDP: Transport Layer Actions



UDP: Transport Layer Actions

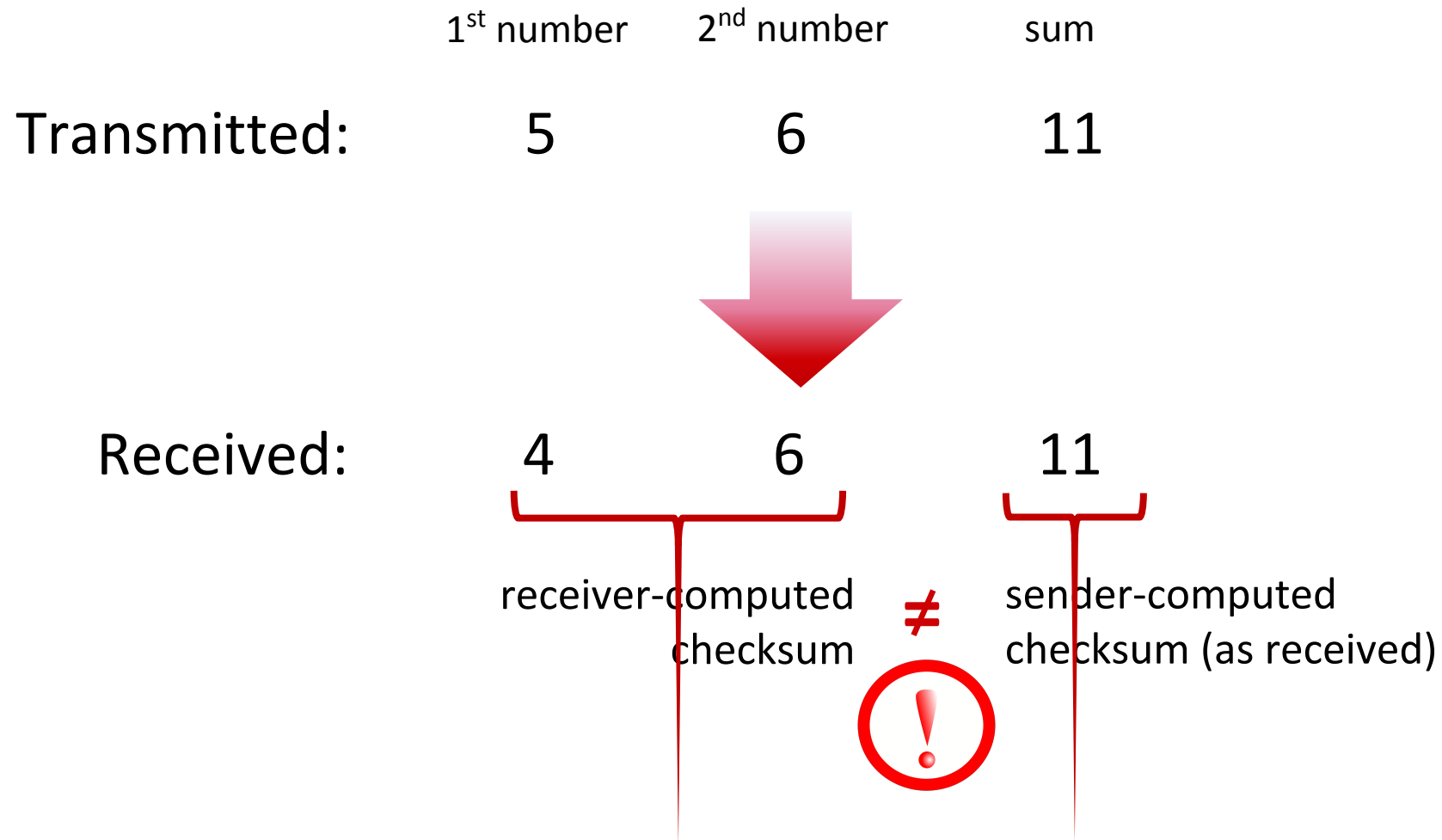


UDP segment header



UDP checksum

Goal: detect errors (*i.e.*, flipped bits) in transmitted segment



UDP checksum

Goal: detect errors (*i.e.*, flipped bits) in transmitted segment

sender:

- treat contents of UDP segment (including UDP header fields and IP addresses) as sequence of 16-bit integers
- **checksum:** addition (one's complement sum) of segment content
- checksum value put into UDP checksum field

receiver:

- compute checksum of received segment
- check if computed checksum equals checksum field value:
 - Not equal - error detected
 - Equal - no error detected. *But maybe errors nonetheless?* More later

Summary: UDP

- “no frills” protocol:
 - segments may be lost, delivered out of order
 - best effort service: “send and hope for the best”
- UDP has its plusses:
 - no setup/handshaking needed (no RTT incurred)
 - can function when network service is compromised
 - helps with reliability (checksum)
- build additional functionality on top of UDP in application layer (e.g., HTTP/3)

Chapter 3: roadmap

- Transport-layer services
- Connectionless transport: UDP
- **Principles of reliable data transfer**
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality



Principles of reliable data transfer

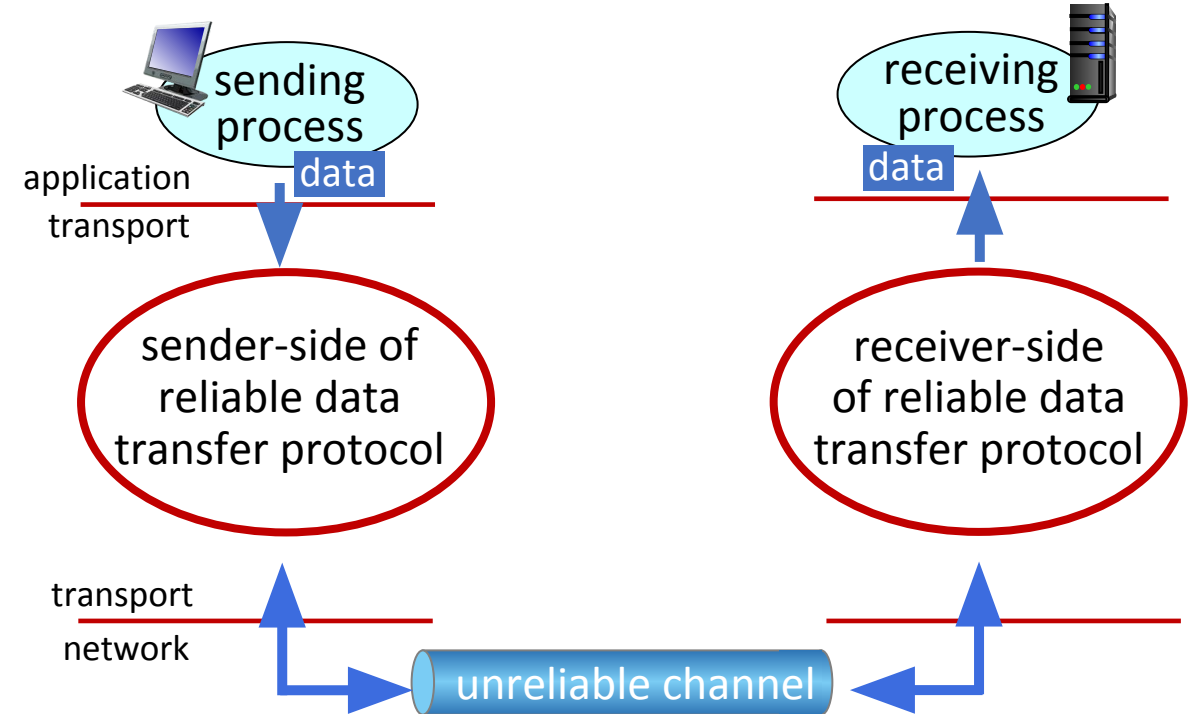
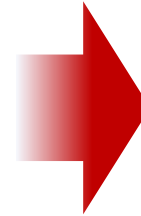


reliable service *abstraction*

Principles of reliable data transfer



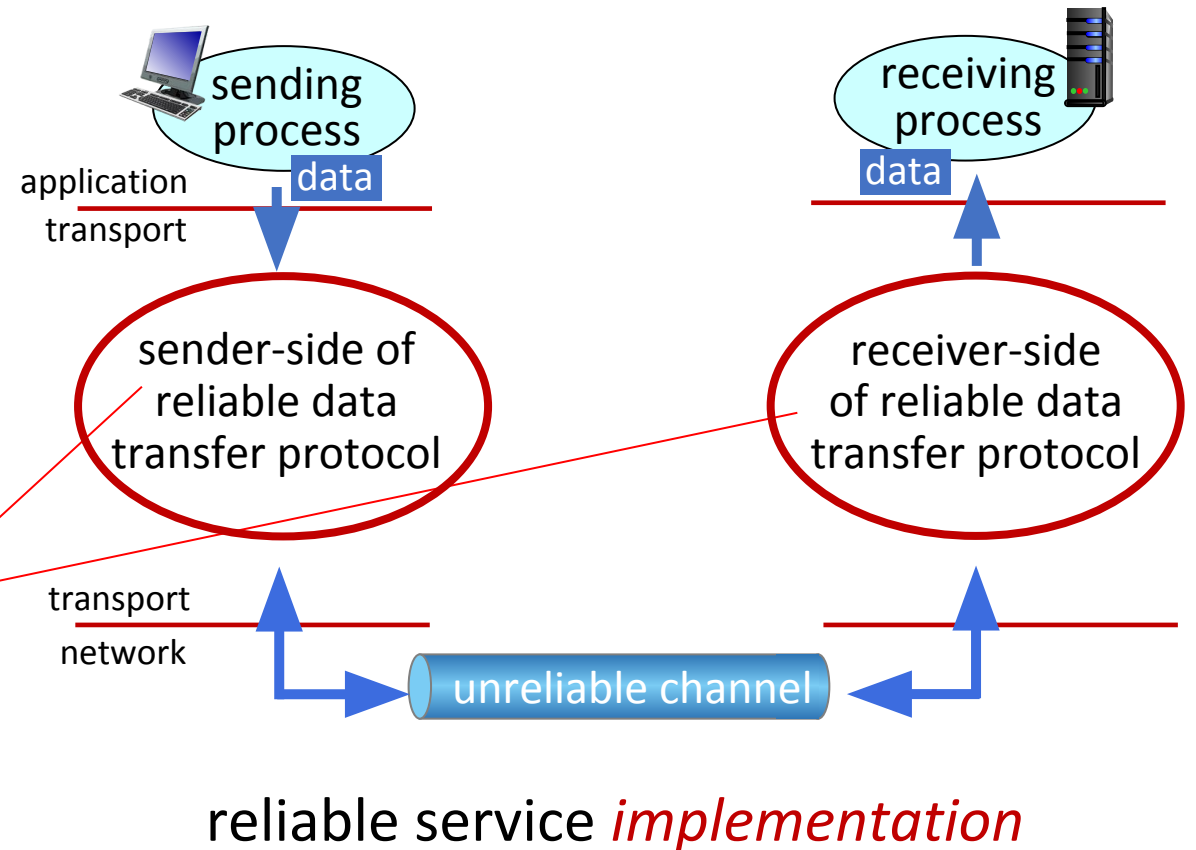
reliable service *abstraction*



reliable service *implementation*

Principles of reliable data transfer

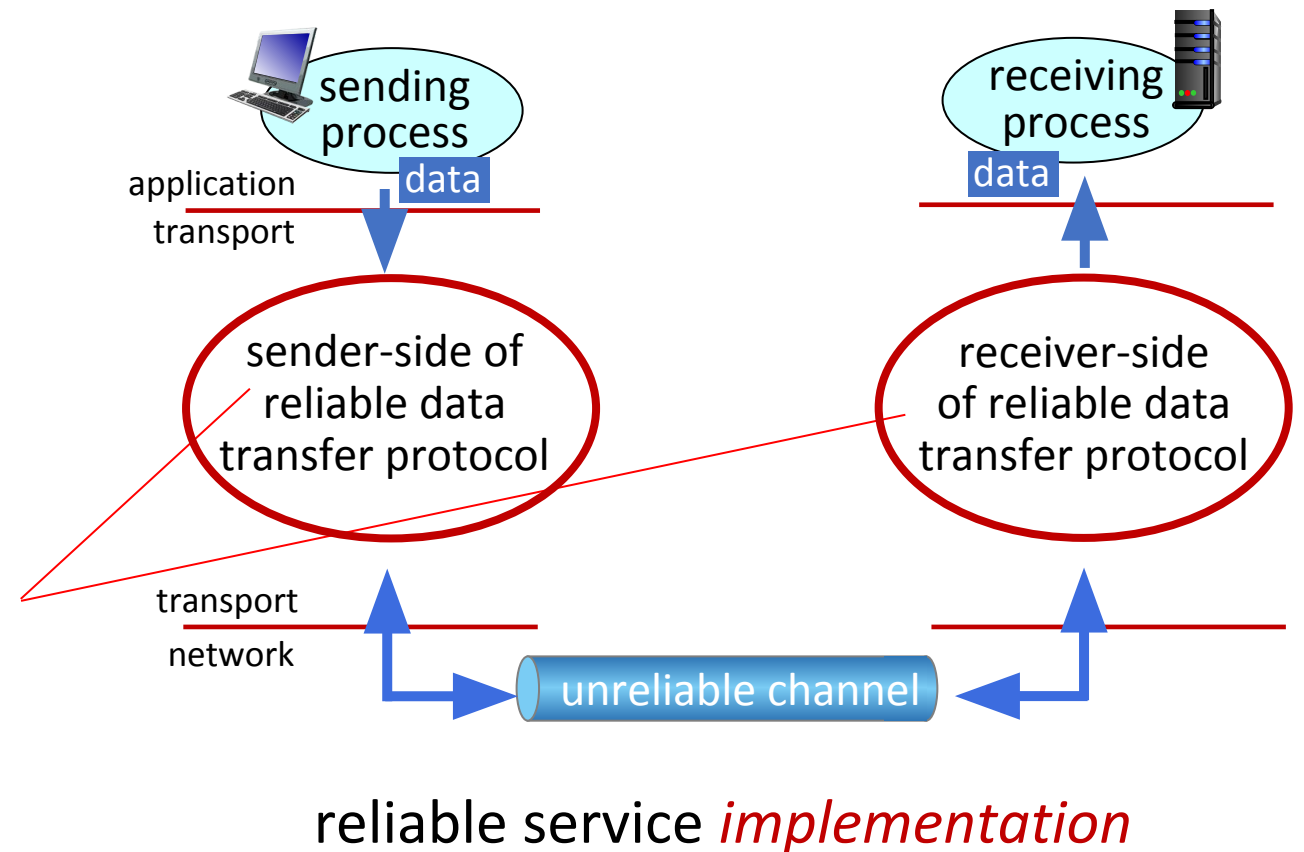
Complexity of reliable data transfer protocol will depend (strongly) on characteristics of unreliable channel (lose, corrupt, re-order data?)



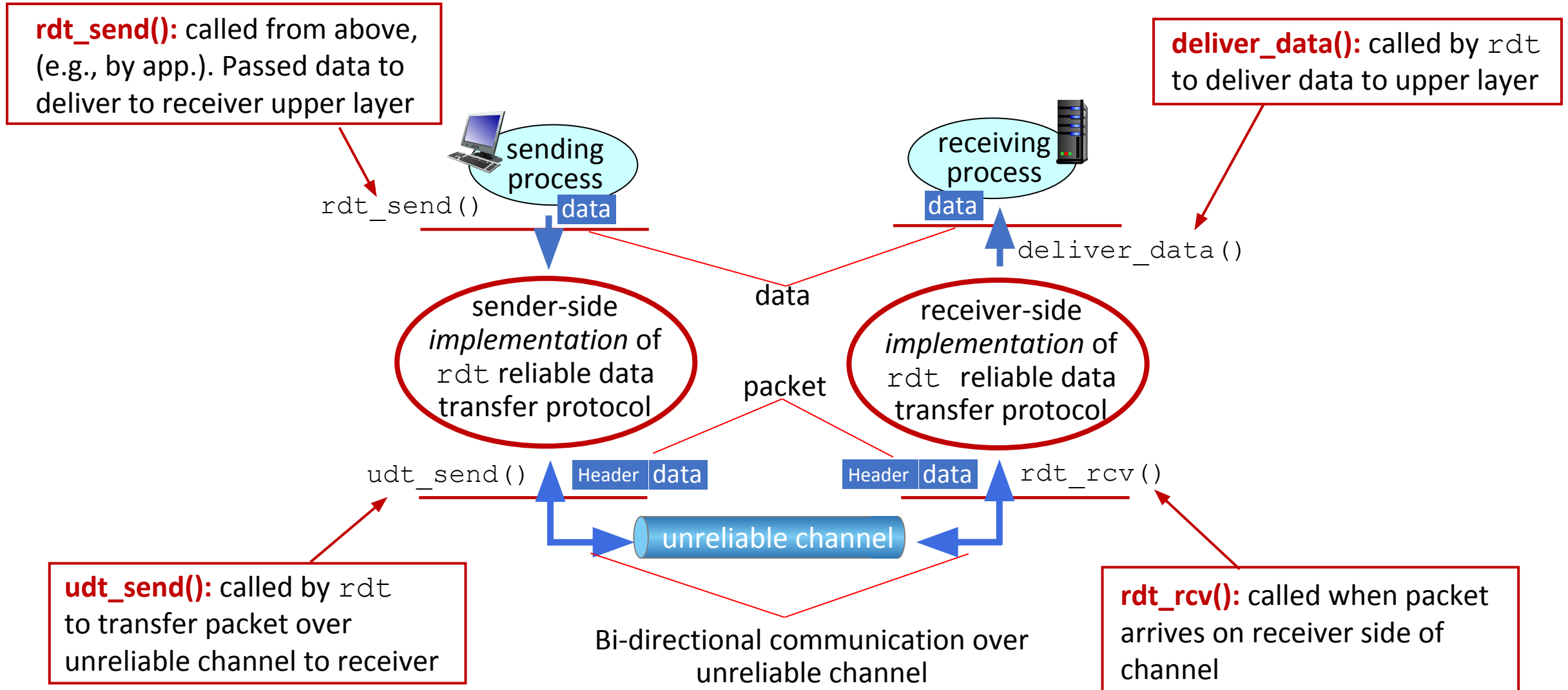
Principles of reliable data transfer

Sender, receiver do *not* know the “state” of each other, e.g., was a message received?

- unless communicated via a message



Reliable data transfer (rdt) protocol: interfaces



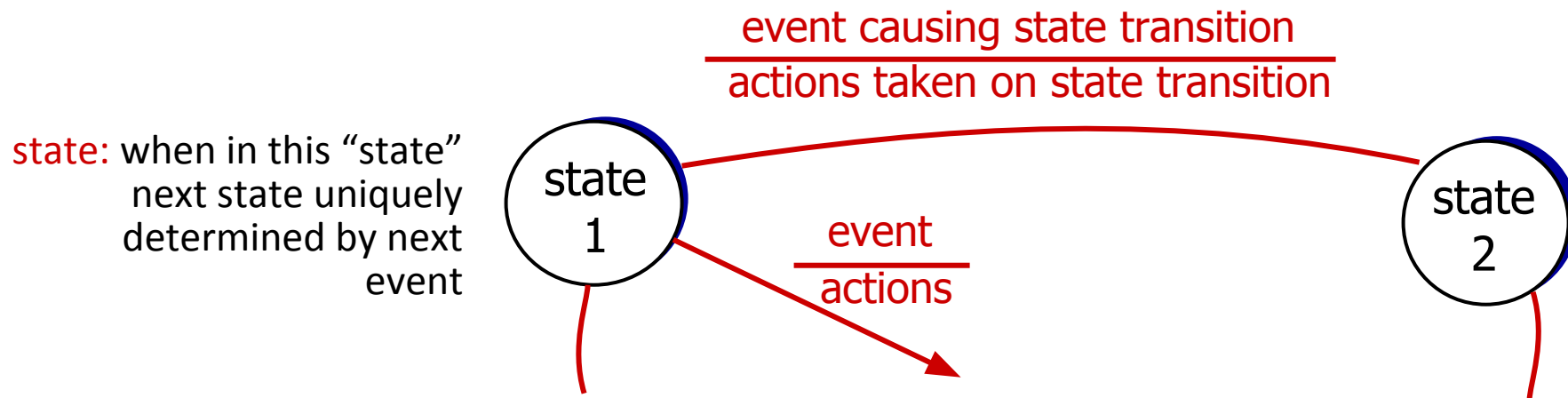
Principles of Reliable Data Transfer in UDP and Protocols

- UDP itself **does not provide reliability**, but applications can build reliability on top of UDP.
- Kurose & Ross introduce RDT protocols to explain how reliability is achieved using:
 - **Checksums**: detect errors
 - **Sequence numbers**: detect lost or duplicate packets
 - **ACKs**: acknowledgments from receiver
 - **Timers**: to re-transmit lost packets

Reliable data transfer (rdt): getting started

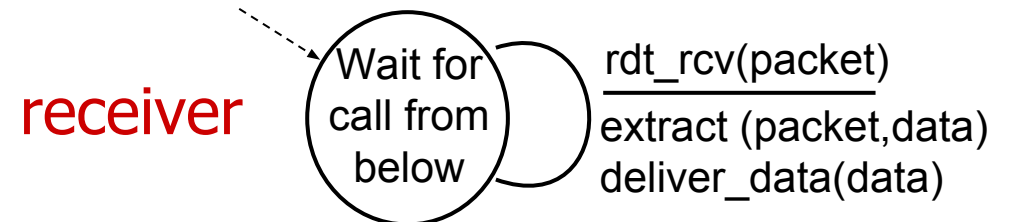
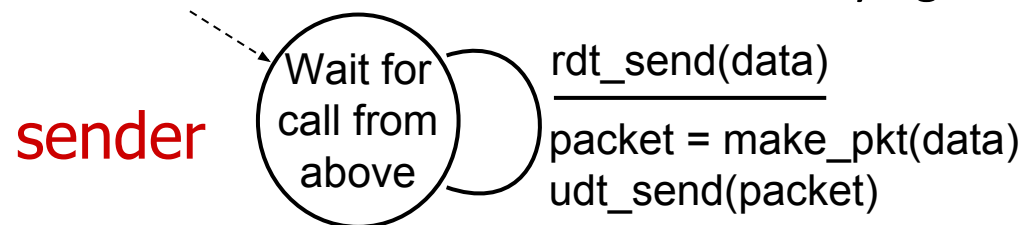
We will:

- incrementally develop sender, receiver sides of reliable data transfer protocol (rdt).
- consider only unidirectional data transfer
 - but control info will flow in both directions!
- use finite state machines (FSM) to specify sender, receiver



rdt1.0: reliable transfer over a reliable channel

- **Assumptions:** No bit errors, No packet losses, No packet duplication, No delays.
- **Sender:** Takes data from upper layer and Sends it directly to receiver.
- **Receiver:** Reads data Delivers to upper layer
- No need for ACK, NAK (Negative Acknowledgment), Sequence numbers and Timers
- Very simple model. Everything works perfectly. Not realistic.
- *separate* FSMs for sender, receiver:
 - sender sends data into underlying channel
 - receiver reads data from underlying channel



rdt2.0: channel with bit errors

- Packets can get bit errors during transmission.
- **Example:** 01010110 → 01000110

Sender behavior:

- Sends a packet
- Waits for ACK/NAK
- If ACK → send next packet
- If NAK → retransmit

Receiver behavior:

- Checks checksum
- If OK → send ACK
- If corrupted → send NAK

rdt2.0: channel with bit errors

- underlying channel may flip bits in packet
 - checksum to detect bit errors
- *the question: how to recover from errors?*
 - *acknowledgements (ACKs)*: receiver explicitly tells sender that pkt received OK
 - *negative acknowledgements (NAKs)*: receiver explicitly tells sender that pkt had errors
 - sender *retransmits* pkt on receipt of NAK

— **stop and wait** —

sender sends one packet, then waits for receiver response

rdt2.0: FSM specification

